

In your reply under "Additional UBX Messages for Quality/Accuracy Assessment" you listed a large range of UBX messages we could use. It seems many of these would be reasonably correlated with each other (eg carrier phase quality and diffCorr would be related to solution fix type, HDOP and position error ellipse values would be correlated etc). What would you suggest as a reasonable set of signals which are reasonably independent measures of fix integrity?

Identified independent signal categories spanning geometry, quality, corrections, and fault det 32s

You're making an excellent point. Many of those signals are indeed outputs of the same underlying Kalman filter or derived from the same measurements. Let me think through what's actually *orthogonal*.

Conceptual Framework: What Can Go Wrong?

The independent failure modes are:

Failure Mode	What breaks	Detection approach
Poor geometry	Satellite positions amplify errors	Geometry metric (DOP)
Weak/degraded signals	Noise increases	Signal quality metric
Multipath	Biased pseudoranges	Residuals, carrier consistency
Correction problems	RTK can't resolve ambiguities	Correction status
RF interference	Noise floor raised	Interference detection
Spoofing	Signals are lies	Spoofing detection
Hardware fault	Antenna/receiver broken	Hardware diagnostics
Cross-receiver	Local issue on one unit	Agreement check

Minimal Independent Signal Set

Here's what I'd suggest as a reasonably orthogonal set:

1. Fix Type (from UBX-NAV-PVT)

What it tells you: Whether carrier phase ambiguities were resolved

This is the primary "did it work?" signal. It's a discrete state rather than a continuous metric, but it's fundamental. RTK Fixed means the receiver is confident it resolved the integer ambiguities.

2. Reported Horizontal Accuracy (hAcc from UBX-NAV-PVT)

What it tells you: The receiver's own estimate of position uncertainty

This incorporates geometry, signal quality, and correction quality into a single metric. It's the receiver's "best estimate" under nominal conditions.

Why keep both fix type AND h_accuracy? Because you can have:

- RTK Fixed with varying h_accuracy (1 cm to 5 cm depending on conditions)
- RTK Float with surprisingly low h_accuracy (if geometry is excellent)
- They encode partially different information

3. PDOP (from UBX-NAV-DOP)

What it tells you: Geometric sensitivity to errors

This is genuinely independent of signal quality. You might ask "doesn't h_accuracy already incorporate geometry?" Yes, but h_accuracy assumes the error model is correct. PDOP tells you "if there's an unmodeled error, how much will geometry amplify it?"

Think of it as:

- **h_accuracy** = expected error under nominal conditions
- **PDOP** = multiplier for off-nominal errors

For safety-critical, you care about both.

4. Correction Age (from UBX-NAV-PVT: `diffAge`)

What it tells you: Staleness of RTCM corrections

This is completely independent of the receiver's measurements—it's about the external correction stream. Even with perfect signals and geometry, old corrections cause drift.

5. Jamming Indicator (from UBX-MON-RF: `jamInd`)

What it tells you: RF interference level

This is measured at the RF front-end, before GNSS signal processing. It detects broadband interference that might not show up clearly in individual satellite metrics. Values 0-255, where higher is worse. Typically <20 is clean, >100 is concerning, >200 is severe.

6. Spoofing Detection State (from UBX-NAV-STATUS: `spooftDetState`)

What it tells you: Whether signals appear to be fake

The F9P has built-in spoofing detection that looks for inconsistencies between signals that would indicate manufactured rather than authentic satellite signals. This is detected via different mechanisms than jamming.

7. Aggregated Signal Quality Metric (derived from UBX-NAV-SAT)

What it tells you: Overall signal health

Rather than tracking every satellite's C/No individually, compute a summary metric:

```
rust

struct SignalQualityMetric {
    /// Number of satellites above usable C/N0 threshold (e.g., 30 dB-Hz)
    sats_above_threshold: u8,

    /// Mean C/N0 of satellites used in solution
    mean_cn0: f32,

    /// Minimum C/N0 among used satellites (weakest link)
    min_cn0: f32,
}

impl SignalQualityMetric {
    fn from_nav_sat(msg: &NavSat) -> Self {
        let used_sats: Vec<_> = msg.satellites
            .iter()
            .filter(|s| s.flags.used_in_solution())
            .collect();

        let cn0_values: Vec<f32> = used_sats.iter().map(|s| s.cno as f32).collect();

        Self {
            sats_above_threshold: cn0_values.iter().filter(|&c| c >= 30.0).count() as u8,
            mean_cn0: cn0_values.iter().sum::<f32>() / cn0_values.len() as f32,
            min_cn0: cn0_values.iter().cloned().fold(f32::MAX, f32::min),
        }
    }
}
```

This is partially correlated with `h_accuracy` (better signals → better solution), but it's measuring the input quality rather than output quality. You can have good signals with bad

geometry, or bad signals with good geometry.

8. Antenna Status (from UBX-MON-RF)

What it tells you: Hardware health

- `antStatus`: OK / short / open / unknown
- `antPower`: on / off

This catches hardware failures that might not be obvious from solution quality (e.g., partial antenna short that degrades but doesn't eliminate signals).

For Dual-Receiver: Two Additional Independent Checks

9. Position Agreement

```
rust

let position_diff = (pos_a - pos_b - known_baseline).norm();
let expected_uncertainty = (h_acc_a.powi(2) + h_acc_b.powi(2)).sqrt();
let disagreement_ratio = position_diff / expected_uncertainty;
```

This is your primary cross-validation. It's independent of all single-receiver metrics because it compares two independent measurements of the same truth.

10. Velocity Agreement

```
rust

let velocity_diff = (vel_a - vel_b).norm();
// Should be < 0.02 m/s for a rigid body
```

Velocity is computed differently than position (primarily from Doppler), so velocity disagreement can catch issues that position agreement might miss, and vice versa.

Summary Table: Minimal Orthogonal Set

Signal	Source	Failure mode detected	Independence
Fix type	NAV-PVT	Ambiguity resolution	Filter output state

Signal	Source	Failure mode detected	Independence
h_accuracy	NAV-PVT	Overall solution quality	Filter output variance
PDOP	NAV-DOP	Geometric weakness	Pure geometry
Correction age	NAV-PVT	NTRIP problems	External dependency
Jamming indicator	MON-RF	RF interference	RF front-end
Spoofing state	NAV-STATUS	Signal authenticity	Signal analysis
Signal quality (mean C/No)	NAV-SAT	Signal strength	Input quality
Antenna status	MON-RF	Hardware fault	Hardware monitor
Position agreement	Dual Rx	Any local fault	Cross-validation
Velocity agreement	Dual Rx	Any local fault	Cross-validation

What I'd Drop

From the original long list, these are correlated enough to skip:

- **NAV-COV:** The covariance matrix is just the detailed version of h_accuracy/v_accuracy. Only useful if you need directional uncertainty.
- **NAV-EELL:** Same—error ellipse is the covariance in a different form.
- **HDOP/VDOP/NDOP/EDOP:** If you have PDOP and h_accuracy, these add little. Maybe keep VDOP separately if vertical accuracy matters independently for your application.
- **Per-satellite residuals:** Useful for debugging but the receiver already uses these internally to weight satellites. The aggregate h_accuracy captures the outcome.
- **Individual satellite C/No, qualityInd, locktime:** Roll these up into the aggregate signal quality metric.

Practical Implementation

```
rust

/// Minimal set of orthogonal integrity signals from a single receiver
struct ReceiverIntegritySignals {
    // Solution quality
    fix_type: FixType,
```

```
h_accuracy_m: f32,

// Geometry
pdop: f32,

// Corrections
correction_age_s: f32,

// RF/Hardware
jamming_indicator: u8,      // 0-255
spoofing_state: SpoofingState,
antenna_status: AntennaStatus,

// Signal quality (aggregated)
satellites_used: u8,
mean_cn0: f32,
}

/// Dual-receiver cross-validation
struct CrossValidationSignals {
    position_disagreement_m: f32,
    velocity_disagreement_mps: f32,
    fix_type_agreement: bool,
}

/// Combined integrity assessment
struct IntegrityAssessment {
    receiver_a: ReceiverIntegritySignals,
    receiver_b: ReceiverIntegritySignals,
    cross_validation: CrossValidationSignals,
}
```

This gives you 10 meaningfully independent signals (8 per-receiver, shared between both, plus 2-3 cross-validation) rather than 20+ correlated ones. Each signal catches a different class of failure, and together they provide good coverage of the failure mode space.